CSE499 A

HOW LLM REASONING WORKS??????

WHERE DO WE CHANGE IT WITH QUANTUM METHOD?????

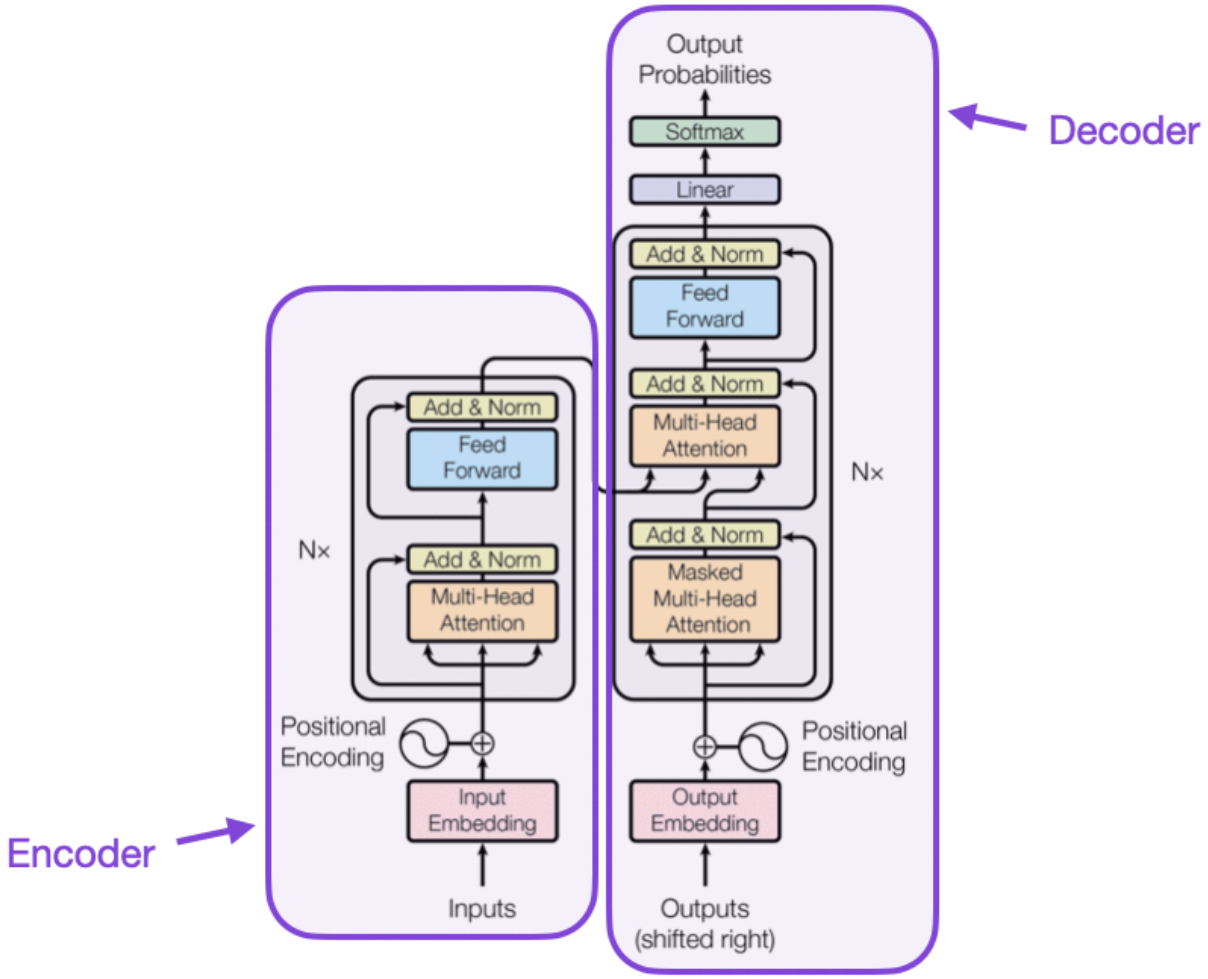


Figure 1: The Transformer - model architecture.

What is an LLM?

A Large Language Model (LLM) is a neural network that models: It predicts the **next token** given previous tokens.

Autoregressive Use

- Generates text **token by token**
- Each new token becomes part of the context
- Inference = repeated probability prediction

Pipeline

Text → Tokenization → Embedding → Transformer Stack → Linear → Softmax → Next Token → (Repeat)

Core Architecture

A) Attention Mechanism

Self-attention computes: Attention(Q,V,K) = softmax(Qk^T/√dk) V

- Q = Query
- K = Key
- V = Value
- d_k = scaling factor

Purpose:

- Learns relationships between tokens
- Maintains global context

B) Positional Encoding

Transformers have no built-in order awareness.

Positional vectors are added:

$z_i = x_i + p_i$

Allows the model to understand sequence order.

C) Feedforward + Normalization

Each layer: Multi-Head Attention

→ Add & Norm

→ Feed Forward

→ Add & Norm

Purpose:

- Nonlinear transformation
- Stabilizes training
- Builds hierarchical representation

Training Workflow

1) Pretraining

- Massive text corpus
- Billions of params
- Masked token for some model
- Next-token prediction
- Self-supervised learning

2) Backpropagation

- Compute gradients
- Update weights via Adam/SGD

3) Fine-Tuning

- Task-specific data
- Instruction tuning

How Reasoning Emerges

(A) Pattern Interpolation

LLMs do **statistical pattern learning**.

They learn:

- Multi-step explanations
- Arithmetic patterns
- Logical sequences

Reasoning = generating learned step patterns.

Not symbolic logic.

Probabilistic structure matching.

(B) Chain-of-Thought (CoT)

Prompt: Let’s think step by step.

Effect:

- Forces intermediate token generation
- Improves structured reasoning

- Reshapes hidden states progressively

Example Reasoning Workflow

Question: "If Sara buys 2 shirts at $20 each and 1 pair of jeans at $50 with 10% discount, how much does she spend?"

Step 1 — Prompt with CoT

Let's think step by step.

Step 2 — Intermediate Token Generation

Instead of jumping to answer, model generates:

1. Shirts: 2 × 20 = 40
2. Jeans: 1 × 50 = 50
3. Total before discount: 40 + 50 = 90
4. Discount: 10% of 90 = 9
5. Final: 90 − 9 = 81

Each step becomes part of the context.

Step 3 — Attention Maintains Context

At every new token:

- Self-attention re-weights **all previous tokens**
- Hidden states update
- Intermediate results influence final probability

Final answer comes from a softmax function.

Where:

- h_t encodes full reasoning history

- Highest probability token = “81”

Key Takeaways

- LLM = next-token probability model
- Transformer = attention-based contextual encoder
- Training = cross-entropy minimization
- Reasoning = structured token generation guided by attention
- CoT improves reasoning by shaping hidden states progressively

Now where do we replace the classical method with the Quantum method???

Now let's first define our project and **project objective**. Our project is to build a **quantum method** for a large language model **reasoning** mechanism. The aim is not to change the whole **LLM architecture** but to change the **classical reasoning mechanism** with the **quantum reasoning mechanism.**

Classical reasoning lives within multiple states but our **quantum reasoning** will live on multiple hypotheses. Makes it more efficient, dynamic, robust and faster than classical methods. Suppose we need to make a drug discovery with the help of LLM. And the ingredient number is 50. Therefore in classical approach 2^{50}-> 10^{15} = will take approximately **31.7 years.** But with our quantum approach the time hypothetically or in theory reduces to 10^{7}, which is faster than a classical reasoning method.

Now enough with the definition and planning and theory but **how do we actually integrate quantum reasoning into a LLM?? Where do we change??? What do we exactly change??**

Inside an LLM reasoning happens during **token selection** (decoding phase).

$$P(x_{t+1}) = Softmax(W_0 h_t)$$

It chooses the next token by collapsing all possibilities except 1 outcome thus forcing **sequential reasoning.**

The model has many possible next tokens but **softmax** collapses to one, even with beam search it only explores a few paths (k << total possibilities). Therefore we can say that **classical reasoning is a narrow path search.**

Our **Quantum reasoning hypothesis** is to exist simultaneously. That maps directly to:

$|\psi> = \sum_t \alpha_i |h_i>$; instead of picking one token, It maintains **superposition of candidate reasoning paths.**

Current Pipeline: **Input** → **Embedding** → **Transformer** → **Hidden State h_t** → **Linear** → **Softmax** → **Next Token.**

Replace this part: **Linear** → **Softmax** → **Token Selection.**

WIth Quantum Module: **Hidden State h_t**

→ **Quantum Hypothesis Encoder**

→ **Superposition of candidate tokens**

→ **Variational Quantum Circuit**

→ **Amplitude Amplification / Rotational Amplification**

→ **Measurement**

→ **Selected token(s)**

What exactly changes? Softmax collapse and greedy or beam search with quantum reasoning module.

1. **Quantum State Encoding:** Convert logits into amplitudes: $|\psi> = \sum_t \sqrt{P_i} |x_i>$

2. **Quantum Reasoning Step:** Variational quantum circuit (VQC) and Encodes constraints / reasoning patterns.

3. **Amplitude Amplification/ Rotational:** Increase probability of valid hypotheses: $\boldsymbol{\alpha}_{correct}\uparrow, \boldsymbol{\alpha}_{wrong}\uparrow$.

4. **Measurement:** Collapse to best token (or small set).

New Reasoning Flow: **Step 1** → superposition of tokens → evolve amplitudes; **Step 2** → refined superposition; **Step 3** → collapse → answer.

Final Architecture:

Input

↓

Embedding

↓

Transformer Stack

↓

Hidden State h_t

↓

Quantum Hypothesis Module

- encode to quantum state

- superposition of hypotheses

- interference + amplification/ Rotation

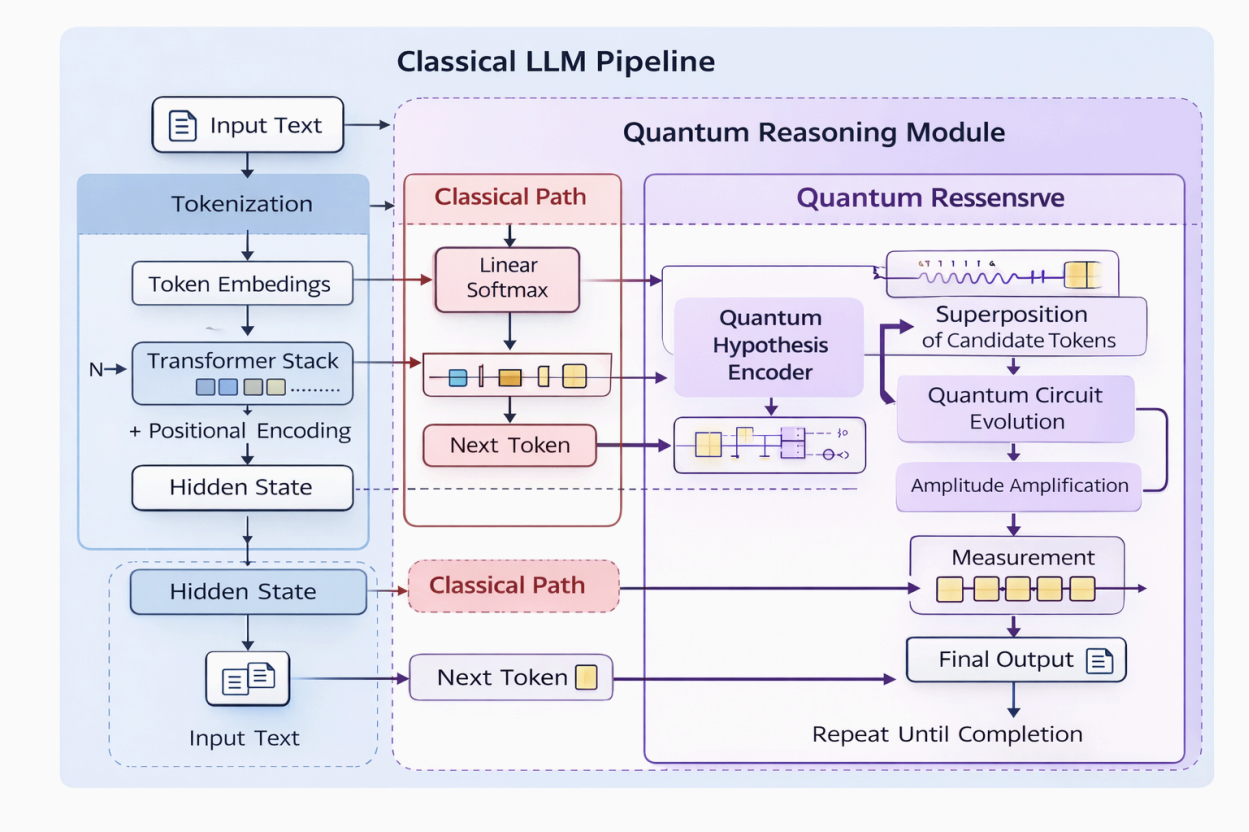
↓

Measurement

↓

Next token(s)

↓

Repeat

FULL SYSTEM VIEW (END-TO-END)